

Codes as Functions

Year 11 Profile Course with Ms. Fernandez and Josh

November 22 2017

1 Introduction

Here we are going to discuss some nice and clean examples involving going from one alphabet to another. The first of these examples will have to do with the types of codes and ciphers that we mentioned last time. The second and third examples will be “linguistic” in nature, meaning they have to do with human natural language. The important thing to remember here is that we are only dealing with *graphemes*, the symbols that make up alphabets, as opposed to *phonemes*, the sounds that may or may not relate to those symbols.¹

2 Codes with the Same Alphabet

What we have been doing up to now is making codes with the same alphabet—in other words, we aren’t adding any new or foreign symbols and we aren’t removing symbols to be restored later.² So let’s look at a cipher from last week where every letter is shifted over by two and our letters wrap around when we run out (our 2-shift cipher if you remember). A super useful option when we want to visualize a function is to write it as a table:

Message Space																									
a	b	c	d	e	f	g	h	i	j	k	l	m	n	o	p	q	r	s	t	u	v	w	x	y	z
↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓
c	d	e	f	g	h	i	j	k	l	m	n	o	p	q	r	s	t	u	v	w	x	y	z	a	b
2-Shift Cipher Space																									

Notice how this isn’t very efficient: there are plenty of other ways of writing a 2-shift cipher—what methods can you think of? How can you take advantage of the structure of our alphabet? (Maybe indices would be useful?)

Now let’s see what happens when we just map every letter to a random other letter:

Message Space																									
a	b	c	d	e	f	g	h	i	j	k	l	m	n	o	p	q	r	s	t	u	v	w	x	y	z
↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓
c	m	j	b	t	x	r	v	d	l	q	a	w	k	z	f	i	o	p	n	g	s	y	e	u	h
Random (Permutation ³) Cipher Space																									

What happens when we put a word through both of these *encrypt()* functions? This idea is called function *composition* where one sticks the output of one function directly into another. We will have more examples on

¹Note that English is notorious for having an orthography (symbolic representation of words) that has very little to do with the phonetics of the words it attempts to depict. For a lovely book introducing linguistics in all of its fantastic glory, I highly recommend *Patterns in the Mind* by Ray Jackendoff (1993), who also happens to be a really nice guy—isn’t it great when people who write great books also happen to be nice people!

²Both of these codes are possible and are extensively used!

³Have a look at what this is online if you’d like: It’s a nice small idea that is often written with horrible notation... think about it for a while, think about all of the stuff we have done so far... notice anything? What does this mean?

this later and go more into depth about what can happen when we do this, but for now the question is: If we compose these encrypt functions (and their associated decrypt functions) and we get back to the original word we started with? Let's go with an example: $2_shift_encrypt(blaba)$ would give us "dncdnc" and putting that through $random_encrypt()$ (so $random_encrypt(2_shift_encrypt(blaba))$) would give us "bkjbkj" in the end. We might think of this as "going backwards" and write our expression the following way:

$$random_decrypt(random_encrypt(2_shift_encrypt(blaba))) = random_decrypt(bkjbkj)$$

but we can also write this as

$$2_shift_decrypt(random_decrypt(bkjbkj)) = 2_shift_decrypt(dncdnc) = blaba$$

Whew... that's a bit ugly... and there are a ton of ways of writing this—explore some for yourself! Try writing this process with our first encrypt function $f()$ and our second encrypt function $g()$ how would it look then? Sometimes, the decrypt function is considered the "inverse" of encrypt (the opposite or reverse operation, we will define this more formally later). We would write these functions as $f^{-1}()$ and $g^{-1}()$ respectively. We will talk about this more in depth later, but give it a shot! What does a simple encrypt-decrypt process look like (resulting in whatever you started with), how about going through both encryption and decryption functions as above?

3 Getting Rid of Useless Letters

One game that I would often play during my high-school English class, was to come up with more efficient English writing schemes.⁴ The goal was not to add any letters, but remove all unnecessary letters. How about "c" as candidate? It only makes the "s" or "k" sounds, so we could just replace "c" with either one of those when it is appropriate right? We are down to 25 baby! How we get there is a bit tricky... someone would have to go through our alphabet and decide when to either get rid of the c's (like before a k) or replace them with "s" or "k" like with *credible* or *scar* or *science* (so we might get *kredible*, *skar*, or *siense*). But wait! Can we go back? Well, our intuition might tell us no—what we are doing is basically mapping 26 letters on to 25... It seems unlikely that 25 letters would have the same amount of information as 26 right? That means that information is "lost" when we go from our original to our modified alphabet!

Original Alphabet																									
a	b	c	d	e	f	g	h	i	j	k	l	m	n	o	p	q	r	s	t	u	v	w	x	y	z
↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓
a	b	s or k	d	e	f	g	h	i	j	k	l	m	n	o	p	q	r	s	t	u	v	w	x	y	z
Modified Alphabet																									

It is worth noting that this is an *encoding scheme* just like the encryption and cipher stuff that we did last week—another way of looking at *all* alphabets is an encoding scheme for the sounds and words that we use in language, how much more relevant to human existence can you get than that?

4 The German and English Alphabets

Let's look at the German alphabet for a bit. This is a way better alphabet (and spelling convention) than English in many ways, even though it has inherited many of the absurdities of various old alphabets like Latin⁵, it's still pretty good. What is interesting is to look at a functions between alphabets, something most German speakers are at least somewhat familiar with when typing.

⁴You should obviously pay attention in English class, but maybe ask your teacher about how we got to the letter system we have today!

⁵Actually Latin wasn't so bad, and most ancient peoples borrowed and adapted alphabets instead of making their own. In fact, the times in history where people have set out to make new alphabets have given historians amazing insight into how old languages may have sounded like, as well as how early different peoples came into contact with each other (or when a given piece of literature made its way into another culture).

So let's make for a very restrictive game, let's take the German alphabet with *äöüß* and try to map the English alphabet onto it, we might get something like this:

En:	a	b	c	d	...	l	m	n	o	p	q	r	s	t	u	v	w	x	y	z	?	?	?	?
↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓
De:	a	b	c	d	...	l	m	n	o	p	q	r	s	t	u	v	w	x	y	z	ä	ö	ü	ß

What do we put at the end? How do we map to our umlauts and eszett? The name of the game is that we can't use combinations of letters to make up for our extra letters in German... "well that's a silly rule!" you might say, "If we do that then we solve the problem!" Well, you are mostly correct but there are some interesting things that happen when we try to replace letters with pairs—namely that we can't easily go back and forth from new to old spellings.⁶ The key here is that with this table, every letter that we take from English has a nice equivalent in German (obviously, as they both use the Latin alphabet), but our eyes (not even our brain!) tells us that we can't cover 30 letters with 26! So our mapping just gives up when faced with a letter it doesn't know—mathematicians might say this mapping is "undefined" for our four German friends. But that doesn't make this function non-interesting! For example, we might notice that for every letter we *can* map to we have only one letter mapping to that letter. Let's say that a different way: if each letter in our German alphabet that has a corresponding English letter has only *one* corresponding English letter...

5 Pulling Everything Together

So, secretly, we were dealing with three different kinds of functions with these examples. Let's go through each of them in a couple lines, here, then next week we will go over these types of functions in more detail, more formally. After that, we will give you some problems to work on that will help solidify this now all-too-amorphous definition of a function.

Surjective Functions In Brief:

In **Getting Rid of Useless Letters** we are definitely able to go from our original to our modified alphabet (the latter of which is one letter smaller than the former) but going backwards, as we discussed, is a problem. However, every letter in the modified alphabet goes back to *at least* one in the original alphabet. This type of relationship between sets (another way of talking about functions) is called *surjective*.

Injective Functions In Brief:

In **The German and English Alphabets** even though every letter in our initial English alphabet mapped nicely to one in the German alphabet, there were some letters left over! A function where each element maps to a *specific* (that means only one!) element is called injective.

Bijective Functions In Brief:

In **Codes with the Same Alphabet** we were looking at what is called *bijective* functions: that is, functions that are *reversible*: where every letter in our message space has a specific letter in our message space *and the other way around*. In order for a function to be bijective it is equivalent to check if it is both surjective and injective—which these first examples in the same alphabet are, can you check and see why?

It is worth noting that neither of our previous two functions were bijective: our injective function was not surjective because there were letters that didn't have partners in our original alphabet, and our surjective function wasn't injective since multiple inputs went to the same output. It is examples like these that we will use to develop our understanding of why these ideas are important and when they are useful.

⁶You might say, this is not a problem, as how often will that have to happen really? Great point! think about it! let's replace "ü" with "ue" for something like Müll this is not a problem (we can go back and forth to Muell) but what about something like *neue*? how do we know not to encode that back to neü?

Where We Are Going:

What we are doing here is actually using functions “intuitively” before defining them with various mathematical notation and terms. We have now briefly covered all of the terms necessary to discuss mathematical functions in their various contexts: *domain* and *range* (alternatively *image* and *preimage*) from our last Cipher handout. The idea of *function composition* which we did relatively early with our *encrypt()* and *decrypt()* functions is as important as that of *recursion* that we have also explored multiple times (sometimes without even mentioning it!). And lastly here we have touched on *injective*, *surjective*, and *bijective* as types or categories of functions. We will explore all of these in the coming couple weeks through some super interesting problems—with ideas ranging super broadly throughout mathematics. If you want to see questions relating to specific topics or ideas then just let us know! In the end, we will be going back to some of the more traditional math questions you will see on standardized tests and in mathematics textbooks, as they are of course also important to your future. But overall we will keep trying to provide concrete and tangible sets of problems so that this abstract concept of functions is as painless (even joyful!) to grok as possible.